

UE5 多人 FPS Demo 技术实现说明

GitHub 链接: [■■■■■■■■■■](#)

Demo 视频链接: [■■■■■■■■■■](#)

1. 项目简介

本项目基于 Unreal Engine 5 官方 First Person 模板开发, 扩展为一个多人 FPS Demo。玩家在竞技场中抵御分波次刷新的敌人, 通过射击击败敌人获得个人分和团队分。当团队分数达到目标值, 或全部波次敌人被清空后, 游戏进入胜利状态。

项目主要覆盖作业要求中的敌人移动与攻击、玩家击败敌人、基础得分与胜利机制、多人网络对战四个核心点。

2. 玩法规则

玩家以第一人称视角进行游戏。关卡开始后, 服务端按波次从场景中的 `AShooterNPCSpawner` 刷新敌人。敌人由 `AIController` 和 `StateTree` 驱动, 感知玩家后追踪并开火攻击。玩家通过武器投射物造成伤害, 敌人生命值归零后死亡。

当前规则配置在 `AShooterGameMode` 中:

- 默认 3 波敌人, 数量为 3、5、8。
- 每击败 1 个敌人, 击杀者获得 10 分, 团队分增加 10 分。
- 团队分达到 120, 或清空所有波次敌人后胜利。
- 胜利状态通过 `AShooterGameState` 复制给所有客户端。

3. 核心模块

3.1 玩家角色

`AShooterCharacter` 继承自 First Person 模板角色, 并保留移动、跳跃、视角控制和武器持有能力。在此基础上增加了多人所需的生命值复制和开火 RPC:

- `CurrentHP` 使用 `ReplicatedUsing=OnRep_CurrentHP` 同步。
- 客户端输入触发 `ServerStartFiring`、`ServerStopFiring` 和 `ServerSwitchWeapon`。

- 武器授予通过服务端执行，避免客户端本地获得武器但服务器不认可。
- 玩家死亡后停止移动、禁用输入并延迟重生。

3.2 敌人 AI

AShooterNPC 是敌人角色，AShooterAIController 负责 AI 控制。模板已有 StateTree、AI 感知和射击任务，本项目将其接入多人规则：

- 敌人只在服务端生成。
- CurrentHP、Weapon、bIsDead 复制到客户端。
- 死亡时调用 AShooterGameMode::HandleEnemyKilled，由服务端统一结算击杀者、个人分和团队分。
- 客户端通过复制状态应用死亡表现，避免不同客户端看到不同敌人状态。

3.3 刷怪与波次

AShooterNPCSpawner 作为关卡中的刷怪点存在。它在 BeginPlay 时向 AShooterGameMode 注册，默认关闭原模板的自循环刷怪，由 GameMode 统一管理波次：

- GameMode 收集所有 Spawner。
- 每波开始时按顺序选择 Spawner 生成敌人。
- 存活敌人数由服务端维护，并复制到 GameState。
- 当前波清空后，延迟进入下一波。

3.4 得分与胜利

多人状态拆分为 GameMode、GameState、PlayerState：

- AShooterGameMode：只在服务端存在，负责规则和胜利判断。
- AShooterGameState：复制团队分数、当前波次、剩余敌人、目标分和胜利状态。
- AShooterPlayerState：复制个人击杀数和个人分数。

敌人死亡后，GameMode 执行：

- 击杀者 AShooterPlayerState::AddKill。
- AShooterGameState::AddTeamScore。
- 更新剩余敌人数。

- 判断是否进入下一波或胜利。

3.5 武器与伤害

AShooterWeapon 和 AShooterProjectile 改为服务端权威：

- 客户端输入通过玩家角色 RPC 请求服务端开火。
- 服务端消耗弹药并生成投射物。
- 投射物设置复制和移动复制。
- 投射物命中后只在服务端调用 ApplyDamage。
- 弹药数通过 RepNotify 更新拥有者 HUD。

3.6 UI 显示

项目新增 AShooterHUD 原生 HUD，用于在多人演示时直接显示关键作业信息：

- Health
- Kills / Score
- Team Score / Target Score
- Wave / Total Waves
- Alive Enemies
- Victory 提示

HUD 数据来自 Pawn、PlayerState 和 GameState，保证 Listen Server 和 Client 窗口都能看到同步后的状态。

4. 多人网络设计

项目采用 Listen Server 模式演示。核心原则是服务端权威、客户端负责输入和显示：

- 玩家移动使用 UE Character Movement 默认复制。
- 玩家射击通过 Server RPC 请求服务端执行。
- 敌人只由服务端生成和驱动 AI。
- 伤害、死亡、得分和胜利全部由服务端计算。
- 全局数据通过 GameState 复制，个人数据通过 PlayerState 复制。

- 敌人和投射物设置 Actor Replication。

这种结构可以减少多人游戏中的状态不一致，并避免客户端直接篡改生命值或分数。

5. 编译验证

已使用 UE 5.7 自带 DotNet 和 UnrealBuildTool 编译游戏目标：

```
TencentFPSDemo Win64 Development
```

结果：Succeeded。

编辑器目标验证时，本机已有 UE 编辑器进程开启且 Live Coding 活跃，UBT 提示需要关闭编辑器或按 `Ctrl+Alt+F11` 后再编译 Editor 目标。

6. 视频录制建议

建议录制 2 到 4 分钟，重点展示：

- 项目基于 First Person 模板。
- 2 个玩家窗口 Listen Server 联机。
- 敌人自动追踪和攻击玩家。
- 玩家击败敌人。
- 个人击杀、团队分和剩余敌人数变化。
- 多波敌人刷新。
- 胜利提示。
- 关键 C++ 类和 GitHub 仓库页面。